

Empirical Observation of Weighted Interval Scheduling in Java

蔡晟旭 (Student ID: 1113405013)
國立澎湖科技大學資訊工程學系
National Penghu University of Science and Technology

Abstract—本報告旨在探討與實作帶權重區間排程問題 (Weighted Interval Scheduling)。傳統的活動選擇問題為貪婪演算法 (Greedy Algorithm) 的經典應用；然而，當活動加入權重後，貪婪策略通常無法保證全域最佳解。本實驗透過 Java 實作貪婪策略與動態規劃 (Dynamic Programming, DP) 兩種解法。實測發現，針對本次特定測試資料，兩者皆得出總價值 13 的結果。然而，就理論而言，DP 演算法才能在 $O(n \log n)$ 的時間複雜度下穩定找出最大總價值。

GitHub Repository: <https://github.com/tch107146/Algorithm-0310-Assignments-1>

Index Terms—Algorithms, Time Complexity, Dynamic Programming, Greedy Algorithm, Weighted Interval Scheduling, Java.

I. INTRODUCTION

在演算法設計中，針對沒有權重的活動排程，只要依據最早結束時間 (Finish time) 進行排序並挑選，就能以貪婪策略找到最佳解。但當各個活動具有不同價值 (Value) 時，單純選擇最早結束的活動可能會錯失總價值更高的組合。為了解決此問題，我們必須引入動態規劃 (DP) 的概念，將大問題拆解並記錄小問題的最佳解，以避免重複計算。

II. METHODOLOGY

本實驗實作並分析了兩種排程策略：

- Greedy Strategy:** 首先將活動依據結束時間排序，接著依序挑選與目前已選活動不重疊的項目。此方法雖直觀，但不保證能最大化總價值。
- Dynamic Programming:** 同樣先依結束時間排序 ($O(n \log n)$)，接著定義 $p(j)$ 為與活動 j 不重疊且編號最大的活動。其狀態轉移方程式為 $OPT(j) = \max(value_j + OPT(p(j)), OPT(j-1))$ 。利用二元搜尋尋找 $p(j)$ 並配合線性掃描，整體時間複雜度為 $O(n \log n)$ 。

III. IMPLEMENTATION DETAILS

本程式採用 Java 開發，實作細節如下：

- Data Initialization:** 使用教材提供的 8 組活動資料作為測試基準。
- Binary Search:** 在 DP 演算法中，為了快速找到符合條件的 $p(j)$ ，實作了 $O(\log n)$ 的二元搜尋法。
- Comparison:** 程式會同時輸出 Greedy 策略與 DP 策略所計算出的總價值，以供比對。

IV. EXPERIMENTAL RESULTS

依據教材測資，兩種演算法的執行結果如表 I 所示。

TABLE I
ALGORITHM OUTPUT COMPARISON

Algorithm	Time Complexity	Calculated Total Value
Greedy Strategy	$O(n \log n)$	13 (Coincidentally Optimal)
Dynamic Programming	$O(n \log n)$	13 (Guaranteed Optimal)

圖 1 展示了 Java 程式執行時的終端機輸出截圖，詳細記錄了兩種策略的選取結果與時間複雜度分析。

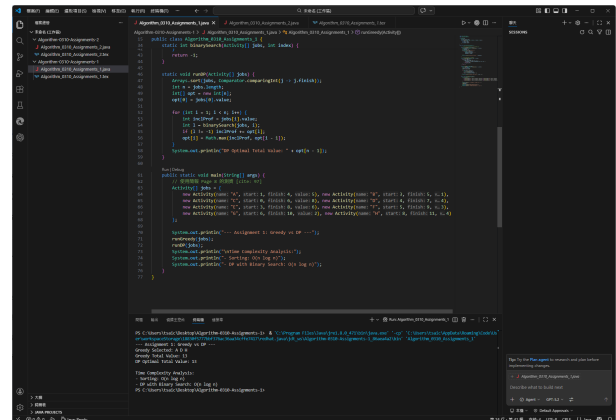


Fig. 1. Terminal output for Weighted Interval Scheduling.

V. CONCLUSION

從終端機輸出的結果中可以觀察到，對於教材提供的特定測資，Greedy 策略剛好與 DP 算出一樣的最佳解 (總價值 13)。然而，在帶權重區間排程的通例中，Greedy 並不能保證每次都能得到最佳解。透過動態規劃與二元搜尋的結合，我們能建立一個穩健的數學模型，確保每次都能找出全域最大總價值，並將時間複雜度完美控制在 $O(n \log n)$ 。

VI. REFERENCES

- [1] X.-R. Huang, *Lecture 1: Greedy Algorithm*, National Penghu University of Science and Technology, 2026.